

AMU | Language, Mind, & Tech
Programming in Python
Ali Asgari



Elephant\in



Print("hello world")



Brian Kernighan's first "Hello World" wasn't a cliché, it was the first time a high-level language (C) successfully bridged the gap between human thought and silicon

sep=	The "Glue." Determines what sits between your data points (commas, arrows, tabs).
end=	The "Anchor." Controls if the output jumps to a new line or stays put (great for progress bars).
f-strings	The "High-Definition Lens." Injects variables directly into text for clean, readable research logs.

Print()



```
# A researcher's way to inspect a sentence structure
word_1 = "Language"
word_2 = "Mind"
word_3 = "Technology"

# 1. The High-Definition Lens (f-strings)
print(f"Current Research Focus: {word_1}, {word_2}, and {word_3}.")

# 2. Using the 'Glue' (sep) to simulate a linguistic path
print(word_1, word_2, word_3, sep=" —> ")

# 3. Using the 'Anchor' (end)
print("Scanning dataset...", end=" /n")
print("Scan Complete.")
```



Side Quest!

By default, every `print()` statement starts a new line. In a research script that is processing 5,000 sentences, how could you use the `end` parameter to print a single dot `.` for every sentence on the *same* line, so you can see the progress without scrolling?

Your reward : 1pt

Variables



Category	Keyword	Nature	Structural Distinction
Numeric	int, float, complex	Numbers	Discrete vs. Continuous vs. Imaginary.
Logical	bool	Binary Truth	A specialized integer: True (1) and False (0).
Text	str	Textual	Immutable sequence of Unicode characters.
Sequence	list	Mutable	An ordered collection that can be changed.
	tuple	Immutable	An ordered collection that is "locked" once created.
Mapping	dict	Keyed	Associative arrays mapping unique keys to values.
Set	set	Mutable	Unordered collection of unique items.
	frozenset	Immutable	A set that cannot be modified after creation.
Binary	bytes	Immutable	Fixed sequence of bytes (0-255).
	bytearray	Mutable	A sequence of bytes that can be modified in-place.
	memoryview	Buffer Access	Accesses internal data without copying it (efficiency).
Empty	None	Singleton	The unique "null" object signifying no value.

Variables



1. Truth & Null States

```
is_active = True          # bool: (True/False)
result = None             # NoneType: (The "Empty" state)
```

2. Mapping & Uniqueness

```
# dict: Mapping keys to values
metadata = {"ID": 105, "L1": "Polish"}
# set: Unique items, mutable
unique_chars = {"a", "b", "c"}
# frozenset: Unique items, locked/immutable
fixed_chars = frozenset(["x", "y", "z"])
```

3. Binary & Memory Management

```
# bytes: Immutable (fixed) binary
raw_data = b"LMT_Signal"
# bytearray: Mutable (changeable) binary
mutable_stream = bytearray(b"LMT_Signal")
mutable_stream[0] = 88      # 'L' becomes 'X' (ASCII 88)

# memoryview: Looking at the 'buffer' without copying
view = memoryview(mutable_stream)
print(f"First byte in buffer: {view[0]}")

# Output using f-strings for clarity
print(f"Status: {is_active} | Result: {result}")
print(f"Dictionary: {metadata}")
print(f"Modified Bytearray: {mutable_stream}")
```



If a Variable is a noun, an Expression is a sentence

Any piece of code that Python can evaluate to a **single value** is an expression

The Ingredients	The Process	The Result
<code>10 + 5</code>	Arithmetic	15
<code>score > 0.8</code>	Comparison	True
<code>"Mind" + "Lab"</code>	Concatenation	"MindLab"
<code>is_valid and has_data</code>	Logic	False

Expressions



```
# 1. Calculating a Mean (Arithmetic Expression)
```

```
total_score = 450
```

```
n_participants = 10
```

```
mean_score = total_score / n_participants # Evaluates to 45.0
```

```
# 2. Building a Research Log (String Expression)
```

```
subject_id = "001"
```

```
log_entry = "Subject_" + subject_id # Evaluates to "Subject_001"
```

```
# 3. Setting a Threshold (Relational Expression)
```

```
# Is the mean score higher than our hypothesis?
```

```
hypothesis_met = mean_score > 40 # Evaluates to True
```

```
print(f"Entry: {log_entry} | Mean: {mean_score} | Goal Met: {hypothesis_met}")
```

Operators



Category	Tools (Operators)	Primary Function
Arithmetic	<code>+, -, *, /, //, %, **</code>	Basic math, floor division, remainders, and powers.
Comparison	<code>==, !=, >, <, >=, <=</code>	Testing equality, magnitude, and divergence.
Logical	<code>and, or, not</code>	Building complex "If/Then" research gates.
Assignment	<code>=, +=, -=, *=, /=</code>	Storing or updating laboratory variables in-place.
Identity	<code>is, is not</code>	Checking if two labels point to the exact same memory object.
Membership	<code>in, not in</code>	Finding tokens or patterns within a corpus or dataset.



Arithmetic Operators

```
# Processing 103 tokens into batches of 10
total_tokens = 103
batch_size = 10

full_batches = total_tokens // batch_size # Floor Division: 10
leftover = total_tokens % batch_size      # Modulo (Remainder): 3
power_calc = 2 ** 3                       # Exponent: 8

print(f"Batches: {full_batches}, Remainder: {leftover}")
```



Comparison Operators

```
rt_subject_a = 450.5
rt_subject_b = 450.5
threshold = 500

print(rt_subject_a == rt_subject_b) # True: Values are identical
print(rt_subject_a > threshold)     # False: 450.5 is not greater than 500
print(rt_subject_a != 500)          # True: 450.5 is not equal to 500
```




Assignment Operators

```
score = 0
score += 10  # Add 10 (score is now 10)
score *= 2   # Multiply by 2 (score is now 20)
score /= 5   # Divide by 5 (score is now 4.0)

print(f"Final Score: {score}")
```

Operators



Logical Operators

```
# Rule 1: 'not' is evaluated BEFORE 'and'
# Result: (not False) and False -> True and False -> False
check_1 = not False and False
print(f"Priority 1 (not before and): {check_1}")
```

```
# Rule 2: 'not' is evaluated BEFORE 'or'
# Result: (not False) or False -> True or False -> True
check_2 = not False or False
print(f"Priority 2 (not before or): {check_2}")
```

```
# Rule 3: 'and' is evaluated BEFORE 'or'
# Result: True or (True and False) -> True or False -> True
check_3 = True or True and False
print(f"Priority 3 (and before or): {check_3}")
```

```
# Rule 4: Parentheses override everything
# Result: not (False and True) -> not (False) -> True
check_4 = not (False and True)
print(f"Manual Override (parentheses): {check_4}")
```



Membership & Identity Operators

```
stimuli = ["apple", "banana", "cherry"]
word = "apple"

# Membership: Check if the word exists in the list
is_present = word in stimuli # True

# Identity: Check if 'word' points to the same memory as the list item
is_same_object = word is stimuli[0] # True

print(f"In List: {is_present} | Same Object: {is_same_object}")
```



Side Quest!

Treat the final True as 1/False as 0

`(9 % 2 ** 3 // 1) * (not (False and True) or False) + (11 // 3 % 2) * (not True or not (False or False))`

Your reward : 2pt

Encoding



A string is a sequence of numbers wearing character masks

1. ASCII (The Foundation):

128 characters (English alphabet, numbers, basic symbols).

Size: 7-bit (stored in 1 byte).

No accents, no emojis, no non-Latin scripts.

2. Unicode (The Universal Standard):

Over 140,000 characters (Every language, ancient scripts, emojis).

Goal: One unique number (Code Point) for every character in existence.

3. UTF-8 (The Translator):

The most common "encoding" used on the web and in Python.

Efficiency: Uses 1 byte for English, but up to 4 bytes for complex symbols.

Method

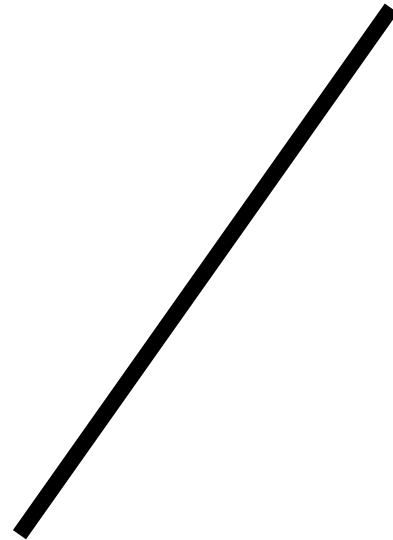
A method is a function
that belongs to a specific object



Function

A general tool

```
print(x)  
len(x)
```



Method

A tool built into
the data type itself

```
x.upper()  
x.lower()
```

String



Single/Double/Triple
Quotes

Text[8]
Text[-1]

Text='Cognition'

Text[0]
Text[-9]

9

```
x=len(text)  
print(x)
```

on

```
Slicing: [start:stop:step]  
y=text[1:5:2]  
print(y)
```

String



```
# 1. Multi-line Transcripts (Triple Quotes)
dialogue = """Subject 01: The stimulus was red.
Subject 02: I disagree, it was orange."""
```

```
# 2. Concatenation & Multiplication
prefix = "Neuro"
field = prefix + "science" # "Neuroscience"
divider = "-" * 10        # "-----"
```

```
# 3. Slicing Geometry [start:stop:step]
word = "Cognition"
# Get 'Cognit' (indices 0 to 5)
root = word[0:6]
# Get every second letter
rhythm = word[::2]      # "Cgnto"
# Reverse the word
reverse = word[::-1]    # "noitingoC"
```

```
# 4. Measuring and Finding (Functions/Operators)
text_length = len(word) # 9
has_ion = "ion" in word  # True
```

String



Category	Method	Description
Case	<code>.lower()</code>	Converts all characters in the string to lowercase.
	<code>.upper()</code>	Converts all characters in the string to uppercase.
	<code>.title()</code>	Capitalizes the first letter of every word.
Clean	<code>.strip()</code>	Removes leading and trailing whitespace (spaces, tabs, newlines).
	<code>.replace(a, b)</code>	Searches for all occurrences of substring a and replaces them with b.
Structure	<code>.split(s)</code>	Breaks a string into a list of strings using s as the separator.
	<code>.join(list)</code>	Merges a list of strings into one string, using the caller as the "glue."
Verify	<code>.isalpha()</code>	Returns True if the string contains only letters.
	<code>.isdigit()</code>	Returns True if the string contains only numbers.
	<code>.isspace()</code>	Returns True if the string contains only whitespace.
Search	<code>.count(s)</code>	Returns the number of non-overlapping occurrences of substring s.
	<code>.find(s)</code>	Returns the lowest index where s is found, or -1 if not present.

String



```
# 1. Normalization (Pre-processing)
raw_input = " Cognitive_Science"
# Cleaning the edges and standardizing case
clean_text = raw_input.strip().lower() # "cognitive_science"

# 2. Tokenization (Split and Join)
sentence = "The mind is fast"
# Breaking into a list of words
tokens = sentence.split(" ") # ['The', 'mind', 'is', 'fast']
# Re-joining with a custom separator
flow = " -> ".join(tokens) # "The -> mind -> is -> fast"
```

```
# 3. Verification & Search
entry = "1250"
print(entry.isdigit()) # True

corpus = "The mind is a lab. The mind is open."
# How many times does 'mind' appear?
print(corpus.lower().count("mind")) # 2

# 4. Search and Anonymize
# Find the position of 'lab'
position = corpus.find("lab") # 14
# Replace sensitive info
safe_corpus = corpus.replace("mind", "[SUBJECT]")
```

Control Flow



Component	Feature	Short Description	Code Template
if	The Gate	Mandatory starting point for selection logic.	if condition: # code block
elif	Alternative	Checks a new condition if previous ones were False.	elif condition: # code block
else	Catch-all	Executes if no conditions in the chain were met.	else : # code block
while	Continuous	Repeats as long as the condition remains True.	while condition: # code block
for	Iterative	Steps through every item in a sequence or collection.	for item in sequence: # code block
break	Exit	Instantly terminates the current loop and jumps to the code after it.	if condition: # code block break
continue	Skip	Stops the current iteration and jumps to the top of the loop for the next cycle.	if condition: # code block continue

Control Flow



```
ids = ["P1", "P2", "ERROR", "P3"]

for participant in ids:
    if participant == "ERROR":
        print("Noise detected. Skipping...")
        continue # Jumps back to the top for "P3"

    print(f"Processing data for: {participant}")
```

```
countdown = 3
while countdown > 0:
    print(f"Syncing... {countdown}")
    countdown -= 1
else:
    print("Lab Sync Complete.")
```


Data Structures



Type	Syntax	Ordered?	Mutable?	Unique?	Use Case in Research
List	[a, b]	Yes	Yes	No	Storing a sequence of reaction times or tokens.
Tuple	(a, b)	Yes	No	No	Fixed data (e.g., GPS coordinates, (lat, long)).
Set	{a, b}	No	Yes	Yes	Finding the unique vocabulary in a text corpus.
Frozenset	frozenset()	No	No	Yes	A fixed, immutable version of a set (rarely used).
Dict	{k: v}	Yes*	Yes	Keys	Storing word frequencies (Word : Count).



1. List: Ordered and Changeable

```
tokens = ["mind", "brain", "mind"]
```

```
tokens.append("logic") # Now: ["mind", "brain", "mind", "logic"]
```

2. Set: Unordered and Unique

```
vocabulary = set(tokens) # Results in: {"mind", "brain", "logic"} (duplicates gone!)
```

3. Tuple: Permanent and Fast

```
coordinates = (52.40, 16.92) # Poznań coordinates - cannot be changed
```

4. Dictionary: Key-Value Mapping

```
word_counts = {"mind": 2, "brain": 1, "logic": 1}
```

```
print(f"The word 'mind' appeared {word_counts['mind']} times.")
```